
AirSense AI

An Air Quality *Decision* Agent

From a PM_{2.5} forecaster to a deployed, agentic, language-driven advisory system

“Is it safe to run outside in Kanpur this evening?”

A complete design and implementation report covering the forecasting brain (spatio-temporal graph neural networks with calibrated uncertainty), the multi-agent decision loop (forecaster, route-planner, notification), and the natural-language interface (tool-using LLM with retrieval-grounded, personalised risk explanations).

Author	Reebal
Programme	B.Tech (3rd year), Indian Institute of Technology Kanpur
Document type	End-to-end project report & implementation plan
Status	Design specification (v1)

June 15, 2026

Abstract

This report describes, end to end, the design and planned implementation of **AirSense AI**: an air-quality *decision* agent that turns a trained $\text{PM}_{2.5}$ forecasting model into a deployed system that actually helps a person decide what to do. The starting point is an existing $\text{PM}_{2.5}$ forecasting pipeline (data cleaning, station clustering, and tree-ensemble regression). The project upgrades that pipeline along three axes and wraps the result in an agentic, language-driven product.

(1) A better brain. The point forecaster is upgraded into a *spatio-temporal, probabilistic* model. Spatial structure is captured by treating the monitoring network as a *graph* and using graph neural networks (GCN, GAT, and spatio-temporal variants such as STGCN, DCRNN and Graph WaveNet). Uncertainty is captured by a *probabilistic head* (Gaussian likelihood and/or quantile regression) and then *calibrated* using conformal prediction, so the system reports honest confidence intervals rather than a single number.

(2) An agent loop. Three cooperating agents sit on top of the model. A *forecaster agent* ingests live measurements and weather and produces the forecast field. A *route-planner agent* formulates outdoor activity as a shortest-path problem on a space–time graph whose edge costs are *inhaled dose*, and optimises route and timing. A *notification agent* decides *when* to alert using a decision-theoretic, learned threshold so that it warns when it matters but does not spam.

(3) A language interface. A tool-using large language model (LLM) provides the natural-language front end. It interprets free-form questions, calls the forecaster and planner as tools, grounds its statements in retrieved health guidance (retrieval-augmented generation), and produces a *personalised* risk explanation conditioned on the user’s health profile, with appropriate medical-safety guardrails.

For every component the report gives the **motivation** (why it is needed), the **mathematics** (the precise formulation, losses, and guarantees), and the **code execution** (illustrative, runnable-style implementations). The report closes with an end-to-end walkthrough of the running example, an evaluation protocol, an MLOps/deployment plan, an ethics and limitations discussion, and a milestone timeline suitable for a mentored student team.

Keywords: air quality, $\text{PM}_{2.5}$, time-series forecasting, spatio-temporal graph neural networks, probabilistic forecasting, conformal prediction, multi-agent systems, LLM tool use, retrieval-augmented generation, decision theory.

Contents

Abstract	1
I Problem, Motivation, and Foundations	5
1 Introduction	6
1.1 The problem we are attacking	6
1.2 From a forecaster to a decision agent	6
1.3 What this project will deliver (“what am I going to do”)	7
1.4 System overview	7
1.5 How to read this report	7
2 Background and Problem Formulation	9
2.1 PM _{2.5} , the AQI, and what counts as “safe”	9
2.2 The forecasting problem, formally	9
2.3 Recap: the existing pipeline (call it v0)	10
2.4 Why v0 is not enough (the gap that motivates v1)	10
II The Forecasting Brain	11
3 Spatio-temporal Data and Graph Construction	12
3.1 The monitoring network as a graph	12
3.2 How to build the adjacency matrix	12
3.3 Normalisation and windowing	13
4 Temporal Modelling	14
4.1 Recurrent networks: LSTM and GRU	14
4.2 Temporal convolutions (TCN)	14
4.3 Attention	15
5 Graph Neural Networks	16
5.1 Message passing: the unifying view	16
5.2 Spectral derivation of the GCN	16
5.3 Graph attention (GAT)	17
5.4 Spatio-temporal architectures (what we will implement)	17
6 Probabilistic Forecasting and Calibration	19
6.1 Why a point forecast is not enough	19
6.2 Parametric head: heteroscedastic Gaussian	19
6.3 Quantile regression and the pinball loss	19

6.4	Conformal prediction: intervals with a guarantee	20
6.5	Scoring rules: how to grade a probabilistic forecast	21
7	Training and Evaluation Protocol	22
7.1	Losses and optimisation	22
7.2	The out-of-time discipline (non-negotiable)	22
7.3	Metrics and baselines	22
7.4	Ablations (the experiments that make it a project, not a demo)	23
III	The Agentic Decision System	24
8	Agents: Concepts and Orchestration	25
8.1	What “agent” means here	25
8.2	Orchestration: a shared-state graph	25
9	The Forecaster Agent	27
9.1	Job and motivation	27
9.2	Data ingestion	27
9.3	Serving and scheduling	28
10	The Route-Planner Agent	29
10.1	Job and motivation	29
10.2	Exposure as a path integral	29
10.3	Discretisation: a space–time graph and shortest paths	29
10.4	Multi-objective: time vs. exposure	30
11	The Notification Agent	32
11.1	Job and motivation: warn when it matters, never spam	32
11.2	Decision-theoretic threshold	32
11.3	Learning the threshold online (and personalising it)	33
IV	The Natural-Language Interface	35
12	The LLM Layer	36
12.1	Role: the LLM is the interface, not the forecaster	36
12.2	Tool use (function calling)	36
12.3	Retrieval-augmented generation (grounding)	37
12.4	Personalised risk explanation	38
12.5	Prompt design, structured output, and guardrails	38
V	Integration, Evaluation, and Deployment	40
13	End-to-End Walkthrough and System Architecture	41
13.1	Tracing the running example	41
13.2	Tech stack	42
14	Evaluation Plan (whole system)	43
15	Deployment, Monitoring, and MLOps	44
15.1	Serving and freshness	44
15.2	Drift and calibration monitoring	44

16 Risks, Limitations, and Ethics	45
17 Project Plan, Milestones, and Team	46
18 Conclusion	47
A Notation	50

Part I

Problem, Motivation, and Foundations

Chapter 1

Introduction

1.1 The problem we are attacking

Fine particulate matter with aerodynamic diameter below $2.5\text{ }\mu\text{m}$ — written $\text{PM}_{2.5}$ — is the single most damaging routine air pollutant for human health. Particles this small bypass the upper-airway defences, deposit deep in the alveoli, and cross into the bloodstream, driving cardiovascular and respiratory disease. The World Health Organization’s 2021 air-quality guideline sets the recommended annual mean at $5\text{ }\mu\text{g}/\text{m}^3$ and the 24-hour mean at $15\text{ }\mu\text{g}/\text{m}^3$. India’s National Ambient Air Quality Standards (NAAQS) are looser — $40\text{ }\mu\text{g}/\text{m}^3$ annual and $60\text{ }\mu\text{g}/\text{m}^3$ over 24 hours — and in much of the Indo-Gangetic plain, including Kanpur and Delhi, even those are routinely exceeded by large factors during winter. Concentrations are highly variable in *space* (street to street, neighbourhood to neighbourhood) and in *time* (hour to hour, driven by traffic, meteorology, and crop-residue burning).

A person who wants to exercise, commute, or send a child to school outdoors faces a concrete decision under uncertainty: *given the forecast, is this activity, at this time, on this route, worth the exposure for me?* A raw number on a dashboard does not answer that. The number is impersonal (it ignores the user’s asthma or age), it is point-valued (it hides how uncertain the forecast is), it is not spatial (it is one station, not the user’s route), and it requires the user to do the interpretation. AirSense AI is built to close exactly that gap.

Motivation

A forecasting model is a means, not an end. The value to a human is realised only when the forecast is turned into a *recommendation about an action*: when to run, which route to take, whether to be alerted. This report is about building the machinery that makes that turn — correctly, with calibrated uncertainty, and in plain language.

1.2 From a forecaster to a decision agent

Our existing work produces a $\text{PM}_{2.5}$ *forecast*. This project produces a *decision agent*. The distinction is the organising idea of the whole report and is worth stating sharply.

- A **forecaster** answers: *What will $\text{PM}_{2.5}$ be?* It outputs numbers.
- A **decision agent** answers: *What should you do about it?* It perceives the world (live data), reasons over the forecast and the user’s context, and takes actions (recommends a route, schedules an activity, fires or suppresses an alert, explains the reasoning).

The model we already trained becomes the *brain* of an agentic control loop. Everything else — ingestion,

planning, alerting, language — is built so that the brain’s predictions are converted into good actions for a specific person.

1.3 What this project will deliver (“what am I going to do”)

Concretely, over the course of the project the team will build and ship the following.

1. An upgraded forecasting model (the brain).

- Re-frame the per-station problem as a *spatio-temporal* one over the monitoring network represented as a graph (Chapters 3–5).
- Add a *probabilistic head* producing prediction intervals, and *calibrate* those intervals with conformal prediction (Chapter 6).
- Establish a rigorous training and out-of-time evaluation protocol with proper scoring rules (Chapter 7).

2. A three-agent decision loop.

- A *forecaster agent* that pulls live data, runs the model, and publishes the forecast field (Chapter 9).
- A *route-planner agent* that optimises outdoor routes and timing to minimise inhaled dose (Chapter 10).
- A *notification agent* with a learned, non-spammy alerting threshold (Chapter 11).

3. A natural-language interface. A tool-using LLM that answers free-form questions, grounds itself in retrieved guidance, and gives *personalised* risk explanations with safety guardrails (Chapter 12).

4. An integrated, deployable system: serving, scheduling, storage, monitoring, drift detection, and a clear evaluation harness (Chapters 13–15).

1.4 System overview

Figure 1.1 shows the architecture. Read it as a loop. Live sources feed the **forecaster agent**, which writes a probabilistic forecast field into a shared **state**. The **route-planner** and **notification** agents consume that state to produce route suggestions and alert decisions. The **LLM interface** sits between the user and the agents: it parses the user’s question, calls the agents as *tools*, retrieves health guidance from a knowledge base, and returns a grounded, personalised answer.

1.5 How to read this report

The report is organised in five parts. Part I (this part) sets up the problem and formal notation. Part II builds the forecasting brain. Part III builds the agent loop. Part IV builds the language interface. Part V integrates, evaluates, and deploys the system, and lays out the plan. Throughout, recurring boxes mark the **motivation**, the **key idea**, and any **caution**. Code listings are illustrative PyTorch / Python and are meant to be the skeleton of the real implementation, not pseudo-code.

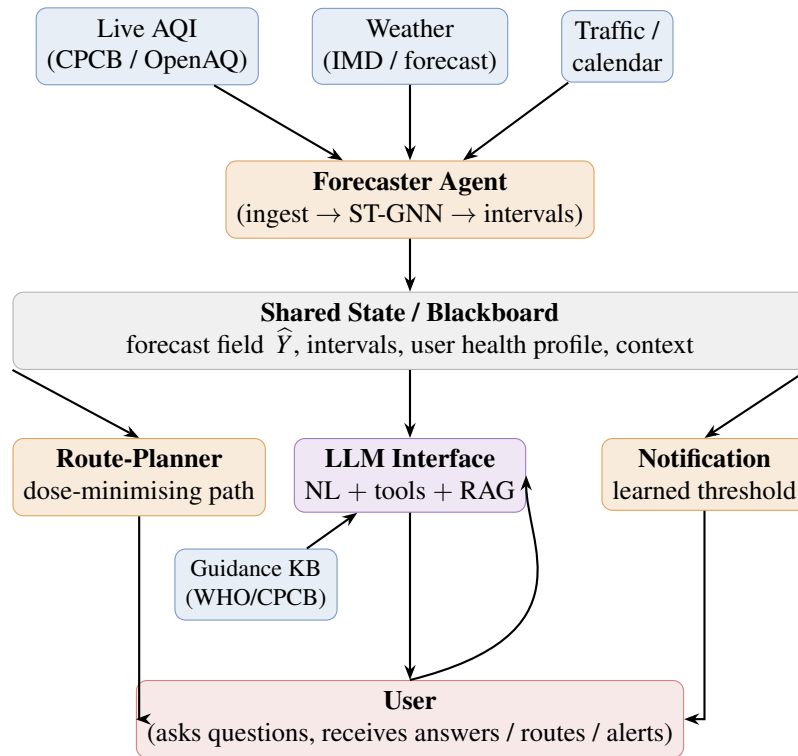


Figure 1.1: AirSense AI architecture. Blue = data, green/orange = ML model and agents, purple = language layer, grey = shared state, red = user. The arrows form a perceive–reason–act loop.

Chapter 2

Background and Problem Formulation

2.1 PM_{2.5}, the AQI, and what counts as “safe”

The Air Quality Index (AQI) maps a raw pollutant concentration to a bounded, colour-coded severity scale through a piecewise-linear interpolation between official breakpoints. For a measured concentration C that falls in a band $[C_{lo}, C_{hi}]$ with corresponding index endpoints $[I_{lo}, I_{hi}]$, the sub-index is

$$\text{AQI}(C) = I_{lo} + \frac{I_{hi} - I_{lo}}{C_{hi} - C_{lo}} (C - C_{lo}). \quad (2.1)$$

The overall AQI is the maximum of the sub-indices over pollutants; for our purposes PM_{2.5} usually dominates in the relevant season. The reason to compute the AQI *as well as* the raw concentration is that the breakpoints encode public-health thresholds (“good”, “moderate”, “poor”, “severe”), and the notification and explanation logic is most naturally expressed in those categorical terms while the model itself predicts the continuous concentration.

Key idea

Predict the continuous quantity PM_{2.5} (it is smoother and carries more information for learning), and convert to AQI categories *at the decision boundary* using Eq. (2.1). Never train the regressor on the bucketed index — bucketing throws away gradient information and creates artificial discontinuities.

2.2 The forecasting problem, formally

Let there be N monitoring stations. At each discrete time step t (say hourly) station i has a feature vector $\mathbf{x}_i^{(t)} \in \mathbb{R}^F$ containing at least the PM_{2.5} reading and typically co-pollutants (PM₁₀, NO₂, O₃, CO), meteorology (temperature, relative humidity, wind speed and direction, boundary-layer height), and calendar encodings (hour-of-day, day-of-week, month, holiday). Stack the network state at time t into a matrix

$$X^{(t)} = [\mathbf{x}_1^{(t)}, \dots, \mathbf{x}_N^{(t)}]^\top \in \mathbb{R}^{N \times F}.$$

Given a look-back window of length P , the input tensor is $\mathcal{X}^{(t)} = (X^{(t-P+1)}, \dots, X^{(t)}) \in \mathbb{R}^{P \times N \times F}$. We want to forecast the target PM_{2.5} at every station over a horizon of Q steps,

$$Y^{(t)} = (y^{(t+1)}, \dots, y^{(t+Q)}) \in \mathbb{R}^{Q \times N}, \quad y_i^{(t+h)} = \text{PM}_{2.5} \text{ at station } i \text{ at time } t+h.$$

The learning problem is to find a map f_θ ,

$$\hat{Y}^{(t)} = f_\theta(\mathcal{X}^{(t)}; G), \quad (2.2)$$

where $G = (\mathcal{V}, \mathcal{E})$ is the station graph (Chapter 3). Two structural facts make this hard and interesting:

1. **Temporal dependence.** $y^{(t+h)}$ depends on the recent trajectory and on exogenous drivers (a wind shift clears a basin; a temperature inversion traps pollution).
2. **Spatial dependence.** Stations are not independent. A plume advects from station i to a downwind station j ; nearby stations share emission sources. The dependence is *not* a simple function of Euclidean distance — wind direction and terrain matter — which is exactly why a learned graph is attractive.

For a probabilistic forecast (Chapter 6) we replace the point output by a predictive distribution; e.g. for each (i, h) we output parameters of a distribution $p_\theta(y_i^{(t+h)} | \mathcal{X}^{(t)})$ or a set of predictive quantiles.

2.3 Recap: the existing pipeline (call it v0)

The project builds on a working PM_{2.5} pipeline. It is worth stating what v0 does, because the upgrades in Part II are defined relative to it. v0, applied to Delhi stations, consists of:

1. **Cleaning.** Outlier handling by inter-quartile-range (IQR) clipping — values outside $[Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR}]$ are treated as suspect — followed by temporal interpolation to fill short gaps.
2. **Spatial summarisation.** Dimensionality reduction with PCA, then K -means clustering of stations into a small number of regimes, so that stations behaving alike are grouped.
3. **Regression.** Three tree-based models — a Decision Tree, a Random Forest, and XGBoost — predict PM_{2.5} from the engineered features.
4. **Honest validation.** *Out-of-time* (OOT) validation: train on an earlier period, test on a strictly later one, so that no information leaks backward across time.

Key idea

v0 is a strong, sober baseline. The OOT discipline and the IQR/interpolation cleaning carry over unchanged. The clustering survives as a useful prior on the graph structure (Section 3.2). What changes is the *model class* (Part II) and the *product around it* (Parts III–IV).

2.4 Why v0 is not enough (the gap that motivates v1)

- **No explicit spatial coupling.** Tree ensembles treat each row independently; they do not know that station j is downwind of i . Cross-station information has to be hand-engineered as features. A graph model learns the coupling.
- **Point predictions only.** A Random Forest can give an empirical spread across trees, but it is not a *calibrated* predictive interval, and decisions need calibrated uncertainty (Chapter 6).
- **Not an interface, not a loop.** v0 is a notebook. It does not ingest live data, does not plan routes, does not decide when to alert, and cannot be queried in natural language. Parts III–IV close that gap.

The remainder of the report is the construction of v1 and the system around it.

Part II

The Forecasting Brain

Chapter 3

Spatio-temporal Data and Graph Construction

3.1 The monitoring network as a graph

The central modelling decision of v1 is to stop treating stations as independent rows and instead treat them as *nodes of a graph*. A graph $G = (\mathcal{V}, \mathcal{E})$ has $N = |\mathcal{V}|$ nodes (stations) and weighted edges encoded by an adjacency matrix $A \in \mathbb{R}^{N \times N}$, where $A_{ij} \geq 0$ measures how strongly station i influences station j . The node feature matrix at time t is the $X^{(t)} \in \mathbb{R}^{N \times F}$ defined in Section 2.2.

Motivation

Pollution is a transport phenomenon on a physical domain. Concentration at a point is the result of emission, advection by wind, diffusion, and deposition. A graph is the natural discretisation of “who affects whom” on that domain, and a graph neural network is the natural function class for learning a map that respects that structure — it shares parameters across nodes and propagates information along edges, exactly as the physics propagates mass along the wind.

3.2 How to build the adjacency matrix

There is no single “correct” A ; the choice is a modelling lever. We will support several and compare them empirically (Chapter 7).

(a) Thresholded Gaussian distance kernel. The classic choice (used by STGCN/DCRNN). With d_{ij} the geographic distance between stations and σ the standard deviation of distances,

$$A_{ij} = \begin{cases} \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right) & \text{if } i \neq j \text{ and } \exp(-d_{ij}^2/\sigma^2) \geq \kappa, \\ 0 & \text{otherwise,} \end{cases} \quad (3.1)$$

where κ is a sparsity threshold. Near stations are strongly connected; far ones are cut.

(b) k -nearest-neighbour graph. Connect each node to its k nearest stations. Guarantees bounded degree and a connected-enough graph even when stations are unevenly spaced.

(c) Correlation / similarity graph. Set A_{ij} from the historical Pearson correlation (or dynamic time-warping similarity) between the $\text{PM}_{2.5}$ series of i and j . This captures coupling that distance misses

(two stations near the same highway corridor). The v0 PCA+K-means clustering is a natural prior here: encourage within-cluster edges.

(d) Learned / adaptive adjacency. Let the data choose the graph. Introduce learnable node embeddings $E_1, E_2 \in \mathbb{R}^{N \times c}$ and define a *self-adaptive* adjacency (Graph WaveNet),

$$\tilde{A}_{\text{adp}} = \text{softmax}(\text{ReLU}(E_1 E_2^\top)), \quad (3.2)$$

where the ReLU enforces non-negativity and sparsity, and the row-wise softmax normalises the out-influence of each node. E_1, E_2 are trained by backprop jointly with the rest of the model, so the graph is discovered, not assumed. In practice we use a *mixture*: a fixed physically-motivated graph plus a learned residual graph.

Key idea

Start with (a) for a sanity-check baseline, then add (d). The headline experiment of the project is: *does a learned adjacency beat a distance kernel for PM_{2.5} over the Kanpur/Delhi network?* That is a clean, publishable question and directly tied to the “graph over monitoring stations” upgrade.

3.3 Normalisation and windowing

Two preprocessing steps matter for stability.

Feature scaling. Standardise each feature channel using statistics computed *only on the training period* (to preserve the OOT guarantee): $z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$. PM_{2.5} is right-skewed and non-negative; a $\log(1 + \text{PM}_{2.5})$ transform before standardisation stabilises variance and prevents the model from chasing extreme winter peaks at the expense of everything else. Predictions are mapped back with $\exp(\cdot) - 1$.

Sliding windows. Convert the long multivariate series into supervised (input, target) pairs with a sliding window of look-back P and horizon Q .

Listing 3.1: Building the station graph and supervised windows.

```

1 import numpy as np
2
3 def gaussian_adjacency(coords, kappa=0.1):
4     """coords: (N, 2) lat/lon (or projected x/y in km)."""
5     N = coords.shape[0]
6     # pairwise squared distances
7     diff = coords[:, None, :] - coords[None, :, :]
8     d2 = (diff ** 2).sum(-1) # (N, N)
9     sigma2 = d2[d2 > 0].std() # spread of distances
10    A = np.exp(-d2 / sigma2)
11    A[A < kappa] = 0.0 # sparsify
12    np.fill_diagonal(A, 0.0)
13    return A
14
15 def make_windows(X, target_idx, P, Q):
16     """X: (T, N, F). Returns inputs (B, P, N, F), targets (B, Q, N)."""
17     T = X.shape[0]
18     xs, ys = [], []
19     for t in range(P - 1, T - Q):
20         xs.append(X[t - P + 1 : t + 1]) # (P, N, F)
21         ys.append(X[t + 1 : t + 1 + Q, :, target_idx]) # (Q, N)
22     return np.stack(xs), np.stack(ys)

```

Chapter 4

Temporal Modelling

Before composing space and time, we fix the temporal component. Three function classes are standard; the spatio-temporal models in Chapter 5 reuse them.

4.1 Recurrent networks: LSTM and GRU

A recurrent network processes the sequence one step at a time, maintaining a hidden state that summarises the past. The Long Short-Term Memory (LSTM) cell updates, for input $\mathbf{x}_t$ and previous state $(\mathbf{h}_{t-1}, \mathbf{c}_{t-1})$,

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \quad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad (4.1)$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad \tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \quad (4.2)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (4.3)$$

The forget gate $\mathbf{f}_t$ controls how much of the past cell state is retained, which is what lets the LSTM carry information across long gaps (e.g. a slow-moving inversion). The GRU is a lighter variant with two gates and no separate cell state; it trains faster with similar accuracy and is a good default for this problem.

Motivation

PM_{2.5} has strong daily and weekly periodicity and slow meteorological regimes. A recurrent state is a compact memory of “where we are in the cycle and what the recent trend is.” For pure single-station forecasting an LSTM/GRU baseline is the first thing to beat.

4.2 Temporal convolutions (TCN)

A Temporal Convolutional Network applies 1-D *causal*, *dilated* convolutions along time. Causal means the output at t depends only on inputs at $\leq t$ (no leakage from the future). Dilation ℓ inserts gaps so the receptive field grows exponentially with depth: with kernel size k and L layers of dilation $1, 2, 4, \dots, 2^{L-1}$, the receptive field is $1 + (k-1)(2^L - 1)$. So a handful of layers see days of history. The dilated causal convolution is

$$(h *_{\ell} g)(t) = \sum_{s=0}^{k-1} g(s) h(t - \ell s). \quad (4.4)$$

TCNs are fully parallel over time (unlike RNNs), train fast, and avoid vanishing gradients via residual connections. Graph WaveNet uses exactly such gated dilated TCNs for its temporal axis.

4.3 Attention

Self-attention lets each time step attend directly to every other, with weights computed from the data. With queries $Q = XW_Q$, keys $K = XW_K$, values $V = XW_V$,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (4.5)$$

The $1/\sqrt{d_k}$ scaling keeps the dot products from saturating the softmax. Attention is useful when a forecast depends on a specific past event regardless of how long ago it was (“the last time wind and humidity looked like this”). It is the most expensive of the three but the most flexible; we will use it sparingly (e.g. a single temporal-attention block) and benchmark whether it earns its cost.

Chapter 5

Graph Neural Networks

This chapter is the technical core of the brain. We build up from the general message-passing framework, derive the GCN from spectral graph theory, cover attention on graphs (GAT), and then present the three spatio-temporal architectures the project will implement and compare.

5.1 Message passing: the unifying view

Almost every GNN layer is an instance of *message passing*. Each node aggregates transformed messages from its neighbours and updates its representation:

$$\mathbf{h}_i^{(\ell+1)} = \phi\left(\mathbf{h}_i^{(\ell)}, \bigoplus_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, A_{ij})\right), \quad (5.1)$$

where ψ computes a message, $\bigoplus$ is a permutation-invariant aggregator (sum, mean, max), and ϕ updates the node. Stacking L layers lets information travel L hops — a plume can influence a node L stations downwind. Different choices of $\psi, \bigoplus, \phi$ recover GCN, GraphSAGE, GAT, and so on.

5.2 Spectral derivation of the GCN

The Graph Convolutional Network (Kipf & Welling, 2017) is the workhorse layer; its derivation clarifies *why* it has the form it does.

Graph Laplacian. With degree matrix $D = \text{diag}(\sum_j A_{ij})$, the symmetric normalised Laplacian is

$$L = I - D^{-1/2} A D^{-1/2} = U \Lambda U^\top, \quad (5.2)$$

where the eigenvectors U form a graph Fourier basis and eigenvalues Λ are graph “frequencies.” A graph convolution with filter g_θ is defined in the spectral domain as $g_\theta \star x = U g_\theta(\Lambda) U^\top x$. Computing U is $O(N^3)$ — too expensive.

Chebyshev approximation (ChebNet). Approximate $g_\theta(\Lambda)$ by a truncated expansion in Chebyshev polynomials T_k of the scaled Laplacian $\tilde{L} = \frac{2}{\lambda_{\max}} L - I$:

$$g_\theta \star x \approx \sum_{k=0}^K \theta_k T_k(\tilde{L}) x, \quad T_k(\tilde{L}) = 2\tilde{L} T_{k-1}(\tilde{L}) - T_{k-2}(\tilde{L}). \quad (5.3)$$

This is K -localised (only K hops) and avoids eigendecomposition.

First-order simplification (GCN). Take $K = 1$, $\lambda_{\max} \approx 2$, tie the two parameters, and apply the *renormalisation trick* $\tilde{A} = A + I$, $\tilde{D} = \text{diag}(\sum_j \tilde{A}_{ij})$. The layer collapses to the famous propagation rule

$$H^{(\ell+1)} = \sigma(\hat{A}H^{(\ell)}W^{(\ell)}), \quad \hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}. \quad (5.4)$$

Here $\hat{A}H$ averages each node's features with its neighbours' (the self-loop keeps its own), and $W^{(\ell)}$ is a shared learnable linear map. Equation (5.4) is what we actually implement.

Listing 5.1: A single GCN layer in PyTorch.

```

1 import torch
2 import torch.nn as nn
3
4 class GCNLayer(nn.Module):
5     def __init__(self, in_dim, out_dim):
6         super().__init__()
7         self.lin = nn.Linear(in_dim, out_dim)
8
9     def forward(self, H, A_hat):
10        # H: (B, N, in_dim); A_hat: (N, N) pre-normalised \hat{A}
11        H = torch.einsum('nm,bmf->bnf', A_hat, H) # neighbour aggregation
12        return torch.relu(self.lin(H)) # shared linear map + nonlinearity
13
14 def normalize_adj(A):
15     A = A + torch.eye(A.size(0), device=A.device) # add self-loops: A + I
16     deg = A.sum(1)
17     d_inv_sqrt = deg.pow(-0.5)
18     d_inv_sqrt[torch.isinf(d_inv_sqrt)] = 0.0
19     D_inv_sqrt = torch.diag(d_inv_sqrt)
20     return D_inv_sqrt @ A @ D_inv_sqrt # \hat{A}

```

5.3 Graph attention (GAT)

The GCN weights neighbours by fixed graph structure. A Graph Attention Network learns the weights from the features. For node i and neighbour j , with a shared linear map W and an attention vector $\mathbf{a}$,

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \| W\mathbf{h}_j]), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \quad (5.5)$$

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W\mathbf{h}_j\right). \quad (5.6)$$

Multiple attention heads are concatenated for stability. GAT is attractive here because the right neighbour weighting is *direction- and regime-dependent* (a station matters more when it is upwind), and attention can express that conditioning, whereas a static $\hat{A}$ cannot.

5.4 Spatio-temporal architectures (what we will implement)

The model must process *space and time jointly*. Three designs, in increasing sophistication:

1) STGCN — “sandwich” of temporal and spatial convolutions. Yu et al. (2018) interleave 1-D temporal (gated) convolutions with graph convolutions:

$$[\text{Temporal-Conv}] \rightarrow [\text{Graph-Conv (Eq. 5.4)}] \rightarrow [\text{Temporal-Conv}].$$

Fully convolutional, so fast and parallel. A strong, simple first target.

2) DCRNN — diffusion convolution inside a GRU. Li et al. (2018) model spatial propagation as a *diffusion process* on the (directed) graph. The diffusion convolution with K steps and transition matrices for both directions is

$$\Theta \star_{\mathcal{G}} X = \sum_{k=0}^{K-1} \left(\theta_{k,1} (D_O^{-1} A)^k + \theta_{k,2} (D_I^{-1} A^{\top})^k \right) X, \quad (5.7)$$

where D_O, D_I are out- and in-degree matrices, so the model captures *asymmetric* upwind/downwind influence. This diffusion convolution replaces the matrix multiplications inside a GRU, giving Diffusion-Convolutional Recurrent network. It is typically the most accurate of the three on traffic-style spatio-temporal data and maps cleanly onto advection.

3) Graph WaveNet — learned graph + dilated TCN. Wu et al. (2019) combine the self-adaptive adjacency of Eq. (3.2) with stacked gated dilated temporal convolutions. It needs no predefined graph and handles long horizons well. This is the model that directly answers our headline question about learned adjacency.

Key idea

Plan of attack for the brain: (i) single-station GRU baseline; (ii) STGCN with a Gaussian adjacency; (iii) Graph WaveNet with a learned adjacency; (iv) DCRNN if time permits. Compare under the OOT protocol with the metrics of Chapter 7. Pick the winner and attach the probabilistic head of Chapter 6.

Listing 5.2: STGCN-style spatio-temporal block (schematic).

```

1 class TemporalConv(nn.Module):
2     """Gated 1-D causal conv along the time axis (GLU)."""
3     def __init__(self, c_in, c_out, k=3):
4         super().__init__()
5         self.conv = nn.Conv2d(c_in, 2 * c_out, (1, k)) # over (N, T)
6         self.c_out = c_out
7
8     def forward(self, x): # x: (B, C, N, T)
9         x = nn.functional.pad(x, (k_left := self.conv.kernel_size[1] - 1, 0))
10        p, q = self.conv(x).split(self.c_out, dim=1)
11        return p * torch.sigmoid(q) # gated linear unit
12
13 class STConvBlock(nn.Module):
14     def __init__(self, c_in, c_spatial, c_out):
15         super().__init__()
16         self.t1 = TemporalConv(c_in, c_spatial)
17         self.gcn = GCNLayer(c_spatial, c_spatial)
18         self.t2 = TemporalConv(c_spatial, c_out)
19         self.norm = nn.LayerNorm(c_out)
20
21     def forward(self, x, A_hat): # x: (B, C, N, T)
22         x = self.t1(x)
23         # apply GCN per time-slice
24         B, C, N, T = x.shape
25         h = x.permute(0, 3, 2, 1).reshape(B * T, N, C)
26         h = self.gcn(h, A_hat).reshape(B, T, N, -1).permute(0, 3, 2, 1)
27         x = self.t2(h)
28         return self.norm(x.transpose(1, 3)).transpose(1, 3)

```

Chapter 6

Probabilistic Forecasting and Calibration

6.1 Why a point forecast is not enough

The notification and route decisions are decisions *under uncertainty*. “PM_{2.5} will be 78 µg/m³” is far less useful than “PM_{2.5} will be 78, and we are 90% sure it is between 61 and 99.” If the interval straddles a health threshold, the honest answer is “uncertain — consider waiting,” which only a probabilistic model can give.

Motivation

A decision agent must know what it does not know. Over-confident point forecasts lead to two failure modes the product must avoid: telling someone it is safe when it might not be (a health risk) and crying wolf (alert fatigue). Calibrated uncertainty is the lever that lets the notification threshold and the language layer behave responsibly.

6.2 Parametric head: heteroscedastic Gaussian

The simplest probabilistic output assumes a Gaussian predictive distribution whose mean *and* variance depend on the input (heteroscedastic — variance is larger during volatile winter episodes). The network outputs $\mu_\theta(\cdot)$ and $\log \sigma_\theta^2(\cdot)$ for each (i, h) , and we minimise the negative log-likelihood

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \left[\frac{(y - \mu_\theta)^2}{\sigma_\theta^2} + \log \sigma_\theta^2 \right] + \text{const.} \quad (6.1)$$

Predicting $\log \sigma^2$ (not σ) keeps the variance positive and the optimisation stable. The first term is a precision-weighted squared error; the second penalises claiming small variance.

6.3 Quantile regression and the pinball loss

A distribution-free alternative is to predict a set of quantiles directly. For target quantile level $\tau \in (0, 1)$, the model output $\hat{q}_\tau$ is trained with the *pinball* (quantile) loss

$$\rho_\tau(y, \hat{q}_\tau) = \begin{cases} \tau(y - \hat{q}_\tau) & \text{if } y \geq \hat{q}_\tau, \\ (1 - \tau)(\hat{q}_\tau - y) & \text{if } y < \hat{q}_\tau. \end{cases} \quad (6.2)$$

Minimising $\mathbb{E}[\rho_\tau]$ yields the conditional τ -quantile. Training a head for, say, $\tau \in \{0.05, 0.25, 0.5, 0.75, 0.95\}$ gives a full predictive interval and a median, with no Gaussian assumption. Quantile crossing (a lower quantile predicted above a higher one) is discouraged by a soft penalty or fixed by sorting at inference.

Listing 6.1: Multi-quantile (pinball) loss.

```

1 def pinball_loss(preds, target, quantiles):
2     # preds: (... , Q_levels); target: (... , 1); quantiles: list of taus
3     losses = []
4     for k, tau in enumerate(quantiles):
5         err = target[... , 0] - preds[... , k]
6         losses.append(torch.max(tau * err, (tau - 1.0) * err))
7     return torch.stack(losses, -1).mean()

```

6.4 Conformal prediction: intervals with a guarantee

Neither the Gaussian NLL nor raw quantile heads guarantee that a “90% interval” actually covers the truth 90% of the time — neural nets are often miscalibrated. *Conformal prediction* fixes this with a finite-sample, distribution-free coverage guarantee, using a held-out *calibration* set.

Split conformal (the core idea). Train f on a training split. On a disjoint calibration set of size n , compute nonconformity scores $s_i = |y_i - f(\mathbf{x}_i)|$. For a target miscoverage α (e.g. $\alpha = 0.1$ for 90% coverage), take the empirical quantile

$$\hat{q} = \text{the } \lceil (n+1)(1-\alpha) \rceil\text{-th smallest } s_i. \quad (6.3)$$

The prediction interval for a new point is $[f(\mathbf{x}) - \hat{q}, f(\mathbf{x}) + \hat{q}]$. Under exchangeability, this guarantees

$$\mathbb{P}(y_{\text{new}} \in [f(\mathbf{x}) - \hat{q}, f(\mathbf{x}) + \hat{q}]) \geq 1 - \alpha. \quad (6.4)$$

Conformalised Quantile Regression (CQR). Combine the best of both: train quantile heads $\hat{q}_{\alpha/2}, \hat{q}_{1-\alpha/2}$ (Section 6.3), then conformalise with the score $s_i = \max\{\hat{q}_{\alpha/2}(\mathbf{x}_i) - y_i, y_i - \hat{q}_{1-\alpha/2}(\mathbf{x}_i)\}$ and widen/shrink the interval by its $(1-\alpha)$ quantile. CQR gives intervals that are both *adaptive* (wider when the model is unsure) and *valid* (Eq. 6.4).

Caution

The exchangeability assumption is *violated* by time series and by distribution shift (a new pollution regime). Use a time-series-aware variant — e.g. *adaptive conformal inference* (ACI), which updates α_t online from realised coverage — and re-calibrate on a rolling recent window. Treat the coverage guarantee as approximate in deployment and monitor realised coverage continuously (Chapter 15).

Listing 6.2: Split-conformal calibration and adaptive (online) update.

```

1 import numpy as np
2
3 def split_conformal_q(cal_resid, alpha=0.1):
4     """cal_resid: |y - f(x)| on the calibration set. Returns q-hat."""
5     n = len(cal_resid)
6     level = np.ceil((n + 1) * (1 - alpha)) / n
7     level = min(level, 1.0)
8     return np.quantile(cal_resid, level, method='higher')
9
10 class AdaptiveConformal:
11     """ACI: adjust alpha_t from realised coverage to keep long-run coverage."""
12     def __init__(self, alpha=0.1, gamma=0.01):
13         self.alpha_t, self.target, self.gamma = alpha, alpha, gamma
14
15     def update(self, covered: bool):
16         err = 0.0 if covered else 1.0 # 1 if truth fell outside interval

```

```

17     self.alpha_t += self.gamma * (self.target - err)
18     self.alpha_t = float(np.clip(self.alpha_t, 1e-3, 0.5))
19     return self.alpha_t

```

6.5 Scoring rules: how to grade a probabilistic forecast

We grade distributions, not points, with *proper* scoring rules (minimised in expectation by the true distribution). The headline metric is the Continuous Ranked Probability Score (CRPS), which generalises MAE to distributions: for predictive CDF F and observation y ,

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(z) - \mathbf{1}\{z \geq y\})^2 dz. \quad (6.5)$$

We also report *coverage* (does the 90% interval contain the truth $\approx 90\%$ of the time?), *sharpness* (interval width — narrower is better, *given* calibration), and the Winkler/interval score that trades the two off. A reliability diagram (predicted vs. empirical coverage) is the key diagnostic plot.

Chapter 7

Training and Evaluation Protocol

7.1 Losses and optimisation

The brain is trained end to end. For the point/quantile model the objective is the pinball loss of Eq. (6.2) (or the NLL of Eq. (6.1)) summed over stations, horizons, and a small L2 weight penalty. Optimise with Adam, cosine learning-rate decay, gradient clipping (spatio-temporal models can have exploding gradients through the recurrent/graph stack), early stopping on a temporal validation slice, and curriculum on the horizon (train short horizons first, then extend Q). Masked losses skip missing sensor values so imputation does not pollute the gradient.

7.2 The out-of-time discipline (non-negotiable)

Key idea

Air-quality data is autocorrelated, so an i.i.d. random train/test split *leaks the future into the past* and reports fantasy accuracy. We split strictly by time: train on $[t_0, t_1)$, calibrate on $[t_1, t_2)$, test on $[t_2, t_3)$, with $t_1 < t_2 < t_3$. For model selection we use rolling-origin (walk-forward) cross-validation, refitting as the origin advances. This carries the v0 OOT principle directly into v1.

7.3 Metrics and baselines

Table 7.1: Evaluation metrics. Point metrics grade the median; probabilistic metrics grade the predictive distribution.

Metric	Definition (per (i, h) , averaged)	What it tells us
MAE	$\frac{1}{n} \sum y - \hat{y} $	Typical error magnitude
RMSE	$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	Penalises large misses
MAPE / sMAPE	mean abs. % error	Scale-free; care near zero
CRPS	Eq. (6.5)	Overall distributional skill
Coverage	% of truths inside the $1 - \alpha$ interval	Calibration
Sharpness	mean interval width	Usefulness given calibration
Winkler score	width + miss penalty	Calibration–sharpness trade-off

Baselines to beat, in order: (i) *persistence* ($\hat{y}^{(t+h)} = y^{(t)}$) and *seasonal persistence* (same hour yesterday) — shockingly hard to beat at short horizons; (ii) the v0 XGBoost regressor; (iii) a single-station GRU. A model is only worth its complexity if it beats all three under OOT. Report per-horizon curves (skill decays as h grows) and break out performance by season and by AQI band (does the model fail exactly during severe episodes, when it matters most?).

7.4 Ablations (the experiments that make it a project, not a demo)

- **Graph ablation:** no graph (GRU) vs. Gaussian-distance graph vs. learned graph (Eq. 3.2) vs. correlation graph. *This is the headline result.*
- **Temporal-module ablation:** RNN vs. TCN vs. attention.
- **Uncertainty ablation:** Gaussian NLL vs. quantile vs. CQR, graded by CRPS and coverage.
- **Horizon ablation:** accuracy and coverage as functions of h .

Part III

The Agentic Decision System

Chapter 8

Agents: Concepts and Orchestration

8.1 What “agent” means here

An *agent* is a process that runs a perceive–reason–act loop toward a goal: it observes the environment, maintains state, decides on an action using a policy, acts, and observes the result. In AirSense AI we use a small number of *specialised, cooperating* agents rather than one monolith, because their goals and time-scales differ:

- the **forecaster agent** runs on a schedule (every hour) and is about *perception*;
- the **route-planner agent** runs on demand (when a user asks) and is about *optimisation*;
- the **notification agent** runs continuously and is about *decision under a cost trade-off*.

Key idea

The trained PM_{2.5} model is the *brain*, but it is not itself an agent — it has no goal and takes no action. The agents wrap the brain’s predictions in goals (minimise exposure, avoid spam, answer the question) and turn predictions into actions. “The ML model is the brain of the agent loop” is precisely the design.

8.2 Orchestration: a shared-state graph

The agents coordinate through a **shared state** (a blackboard) and a directed *orchestration graph* (nodes are agents/tools, edges are control flow), exactly the pattern formalised by frameworks such as Lang-Graph. The state is an explicit, typed object passed between nodes; each agent reads what it needs and writes its result back. This makes the system debuggable, testable, and resumable (a long-running plan can checkpoint its state).

Listing 8.1: The shared state object passed between agents.

```
1 from dataclasses import dataclass, field
2 from typing import Optional
3 import numpy as np
4
5 @dataclass
6 class AirSenseState:
7     # --- perception (written by forecaster agent) ---
8     forecast_median: Optional[np.ndarray] = None          # (Q, N)
9     forecast_lo: Optional[np.ndarray] = None              # (Q, N) lower interval
10    forecast_hi: Optional[np.ndarray] = None              # (Q, N) upper interval
11    station_coords: Optional[np.ndarray] = None           # (N, 2)
```

```
12 forecast_time: Optional[str] = None
13 # --- user context ---
14 user_profile: dict = field(default_factory=dict) # age, asthma, activity, sensitivity
15 query: str = ""
16 location: Optional[tuple] = None # (lat, lon)
17 # --- outputs (written by the consuming agents) ---
18 route: Optional[dict] = None # planner output
19 alert_decision: Optional[dict] = None # notification output
20 answer: str = "" # LLM output
```

Chapter 9

The Forecaster Agent

9.1 Job and motivation

The forecaster agent is the perception layer. Its job: on a schedule, *pull live data*, assemble the model input tensor, *run the brain*, and *publish* the probabilistic forecast field to the shared state, with caching and graceful fallback if a source is down.

Motivation

A model trained on historical data is inert. The forecaster agent is what makes the system *live*: it connects the brain to the world in real time, so that “this evening” means the actual upcoming hours, conditioned on the latest measurements and the latest weather forecast.

9.2 Data ingestion

Sources: live PM_{2.5}/AQI from public APIs (e.g. CPCB’s portal or aggregators such as OpenAQ); meteorology and short-range weather forecasts (the meteorological forecast is itself a powerful *exogenous* predictor of dispersion and must be fed forward over the horizon); and optionally traffic and the user’s calendar for context. The agent must handle the realities of sensor data: missing values, stuck sensors, unit mismatches, and API rate limits.

Listing 9.1: Forecaster agent: ingest, predict, publish (with fallback).

```
1 import time
2
3 class ForecasterAgent:
4     def __init__(self, model, adj, scaler, conformal, fetch_aqi, fetch_weather):
5         self.model, self.adj = model, adj
6         self.scaler, self.conformal = scaler, conformal
7         self.fetch_aqi, self.fetch_weather = fetch_aqi, fetch_weather
8         self._cache = None
9
10    def _assemble_input(self):
11        aqi = self.fetch_aqi()          # (P, N, F_aqi); raises on hard failure
12        wx = self.fetch_weather()       # (P+Q, N, F_wx): includes forecast horizon
13        X = self.scaler.transform(concat_features(aqi, wx))
14        return to_tensor(X)
15
16    def step(self, state: "AirSenseState"):
17        try:
18            x = self._assemble_input()
19        except SourceError:
```

```

20         if self._cache is not None:                # graceful degradation
21             state = self._cache; state.stale = True
22             return state
23         raise
24     with torch.no_grad():
25         q = self.model(x, self.adj)                # quantile outputs (Q, N, n_taus)
26         median = q[..., mid_idx].cpu().numpy()
27         lo, hi = self.conformal.intervals(q)         # calibrated bounds
28         state.forecast_median, state.forecast_lo, state.forecast_hi = median, lo, hi
29         state.forecast_time = time.strftime('%Y-%m-%dT%H:%M')
30         self._cache = state
31         return state

```

9.3 Serving and scheduling

The brain is exported (TorchScript / ONNX) and served behind a thin API. A scheduler (cron / a job queue) triggers `step()` hourly; the result is cached so that many user queries within the hour reuse one forecast. Latency budget: a single forward pass over a few hundred stations is milliseconds on CPU, so the bottleneck is data fetching, not inference — hence aggressive caching.

Chapter 10

The Route-Planner Agent

10.1 Job and motivation

Given the forecast field and an origin/destination (or “a 5 km loop from home in the next 3 hours”), the route-planner finds *where* and *when* to go outdoors to minimise harmful exposure, optionally trading off against time/effort.

Motivation

Two runs of equal length can differ several-fold in inhaled pollution depending on the streets and the start time. The forecast already contains that spatial and temporal variation; the planner is what *exploits* it on the user’s behalf. This is where a spatial forecast pays off in a tangible recommendation.

10.2 Exposure as a path integral

The right objective is not average concentration but *inhaled dose*. For a person moving along a path γ over a route, breathing at minute-ventilation rate $V(\cdot)$ (litres/min, higher during exercise), the inhaled dose is the line integral of concentration $\times$ ventilation:

$$\text{Dose}(\gamma) = \int_{\gamma} C(\mathbf{r}(s), t(s)) V(a(s)) dt, \quad (10.1)$$

where $\mathbf{r}(s)$ is position, $t(s)$ is the clock time when the person is there (so the *space–time* forecast is queried, not a snapshot), and $a(s)$ is the activity intensity. Because t advances as the person moves, this is intrinsically a *time-dependent* routing problem.

10.3 Discretisation: a space–time graph and shortest paths

We discretise the city into a routing graph $G_R = (\mathcal{V}_R, \mathcal{E}_R)$ (intersections and street segments, e.g. from OpenStreetMap). The forecast field, defined at monitoring stations, is interpolated to each segment (inverse-distance or kriging). The cost of traversing edge e at departure time t is the discretised dose,

$$w_e(t) = \underbrace{C(e, t)}_{\text{forecast at that place \& time}} \cdot V \cdot \tau_e, \quad (10.2)$$

with τ_e the time to traverse e . Minimising total dose is then a *time-dependent shortest-path* problem, solvable by Dijkstra’s algorithm (non-negative weights — concentrations are non-negative, so Dijkstra is valid) or, with a geographic lower-bound heuristic, A^* .

Proposition 10.1 (Validity of Dijkstra here). *Edge costs in Eq. (10.2) are products of non-negative quantities, hence $w_e(t) \geq 0$. Dijkstra’s algorithm is therefore optimal for the static-cost relaxation, and its time-dependent variant (label-setting on (node, arrival time) states) is optimal when the network satisfies the FIFO property, which holds because waiting never makes you arrive earlier.*

Algorithm 1: Dose-minimising time-dependent route (A^*)

Input: routing graph G_R , origin o , destination d , departure window, forecast $C(\cdot, \cdot)$, ventilation V , heuristic $h(\cdot)$

Output: path minimising inhaled dose (Eq. 10.1)

```

1 push ( $o, t_{\text{start}}$ ) with  $g=0$ , priority  $f=h(o)$  to open set;
2 while open set non-empty do
3   pop state ( $u, t_u$ ) with smallest  $f$ ;
4   if  $u = d$  then
5     return reconstructed path;
6   foreach edge  $e = (u, v)$  do
7      $\tau \leftarrow$  traversal time of  $e$  at  $t_u$ ;
8      $\Delta \leftarrow C(e, t_u) \cdot V \cdot \tau$ ; // dose on this segment, Eq. 10.2
9     if  $g_u + \Delta < g_v$  then
10       $g_v \leftarrow g_u + \Delta$ ;  $f_v \leftarrow g_v + h(v)$ ;
11      set arrival time  $t_v \leftarrow t_u + \tau$ ; push ( $v, t_v$ );

```

10.4 Multi-objective: time vs. exposure

Users care about both dose and duration. We expose the trade-off rather than hard-coding it. Either (i) *scalarise* with a weight λ , minimising $\text{Dose} + \lambda \cdot \text{Time}$ and letting the user pick λ via a “how much detour is acceptable?” control; or (ii) compute the *Pareto front* of non-dominated (dose, time) routes and present a few options (“the cleanest route is 4 min longer and cuts your dose 35%”). The planner also answers the *timing* question by sweeping the departure time over the forecast window and returning the lowest-dose start time — often more impactful than the route itself.

Listing 10.1: Dose-minimising planner over a time-dependent graph.

```

1 import heapq
2
3 def plan_route(graph, origin, dest, t_start, forecast_at, V, lam=0.0, heuristic=None):
4     """forecast_at(edge, t) -> predicted concentration; lam trades off time vs dose."""
5     heuristic = heuristic or (lambda n: 0.0) # 0 -> reduces to Dijkstra
6     g = {origin: 0.0}
7     pq = [(heuristic(origin), origin, t_start, [origin])]
8     seen = set()
9     while pq:
10         f, u, t_u, path = heapq.heappop(pq)
11         if u == dest:
12             return {"path": path, "cost": g[u], "arrival": t_u}
13         if (u, round(t_u, 2)) in seen:
14             continue
15         seen.add((u, round(t_u, 2)))
16         for v, tau in graph.neighbors(u, t_u): # tau = traversal time of (u,v)
17             dose = forecast_at((u, v), t_u) * V * tau
18             cost = dose + lam * tau # scalarised objective
19             ng = g[u] + cost
20             if ng < g.get(v, float("inf")):
21                 g[v] = ng
22                 heapq.heappush(pq, (ng + heuristic(v), v, t_u + tau, path + [v]))

```

```
23     return None
24
25 def best_departure(graph, origin, dest, window, forecast_at, V, step_min=15):
26     """Sweep start times; return the lowest-dose departure."""
27     best = None
28     for t in window_iter(window, step_min):
29         r = plan_route(graph, origin, dest, t, forecast_at, V)
30         if r and (best is None or r["cost"] < best["cost"]):
31             best = {**r, "depart": t}
32     return best
```


Chapter 11

The Notification Agent

11.1 Job and motivation: warn when it matters, never spam

The notification agent watches the forecast stream and decides, at each opportunity, whether to push an alert (“air turning hazardous around 6 pm — move your run earlier”). The whole challenge is the trade-off: too few alerts and the product fails its safety purpose; too many and the user mutes it (*alert fatigue*), which is the same as failing. The threshold must be *learned* and *personalised*, not a fixed AQI cutoff.

Motivation

A fixed “alert if AQI > 200” rule is wrong for everyone: it spams a resident of a chronically polluted city and under-warns a sensitive asthmatic on a relatively clean day. The right trigger depends on the *change*, the *user’s sensitivity*, the *forecast confidence*, and *how recently we last bothered them*.

11.2 Decision-theoretic threshold

Formulate alerting as a binary decision with a loss matrix. Let the user’s true state be “would benefit from this alert” ($H=1$) or not ($H=0$), and the action be alert/stay-silent. Costs: a *missed* beneficial alert costs c_{FN} (health risk, missed opportunity); a *false* alarm costs c_{FP} (annoyance, erosion of trust). By minimising expected loss, the Bayes-optimal rule is to alert iff the posterior probability of benefit exceeds a threshold set purely by the cost ratio:

$$\text{alert} \iff \mathbb{P}(H=1 \mid \text{forecast, context}) > p^*, \quad p^* = \frac{c_{FP}}{c_{FP} + c_{FN}}. \quad (11.1)$$

The probability on the left is exactly what the *probabilistic* forecaster provides: e.g. $\mathbb{P}(\text{PM}_{2.5} \text{ crosses the user’s personal threshold})$ computed from the predictive interval. If false alarms are cheap relative to misses (c_{FN} large), p^* is low and the agent alerts readily; as annoyance cost rises, p^* rises.

Key idea

This is why probabilistic forecasting (Chapter 6) and notification are tied together. The alert decision is a *thresholded posterior probability of crossing a personal limit*, and that posterior is only available because the brain outputs calibrated intervals, not a point.

11.3 Learning the threshold online (and personalising it)

The cost ratio is not known a priori and differs per user, so we *learn* it from feedback (did the user act on the alert? did they mute / mark “not useful”?). This is a *contextual bandit*: at each opportunity the agent observes context $\mathbf{x}$ (forecast change, confidence, user profile, time since last alert), chooses an action (alert / silent), and receives a reward (e.g. +1 acted-upon, 0 ignored, $-c$ muted). We use Thompson sampling over a model of $\mathbb{P}(\text{useful} | \mathbf{x}, \text{alert})$, which balances exploration (occasionally testing a borderline alert) and exploitation, and naturally adapts p^* to the individual.

Alert-fatigue penalty and hysteresis. Two engineering safeguards sit on top of the learned policy. (1) A *refractory penalty*: add a term that raises the effective threshold for a cool-down period after each alert, so the rate is bounded (at most k alerts/day). (2) *Hysteresis / debouncing*: require the predicted crossing to persist for m consecutive forecast updates before firing, and use separate on/off thresholds ($\theta_{\text{on}} > \theta_{\text{off}}$) so a forecast hovering at the boundary does not flap on and off.

Algorithm 2: Notification decision per forecast update

Input: predictive interval for user’s window, user profile, last-alert time, policy π

```

1  $p \leftarrow \mathbb{P}(\text{PM}_{2.5} > \theta_{\text{user}} \text{ during window})$ ; // from calibrated forecast
2  $p^* \leftarrow \pi.\text{THRESHOLD}(\text{context})$ ; // learned, personalised
3 if time since last alert < refractory then
4    $p^* \leftarrow \min(1, p^* + \beta)$ ; // fatigue penalty
5 if  $p > p^*$  and crossing persisted  $\geq m$  updates then
6   emit alert; record (context, action=alert);
7 else
8   stay silent; record (context, action=silent);
9 on user feedback (acted / ignored / muted):  $\pi.\text{UPDATE}(\text{context}, \text{reward})$ ;

```

Listing 11.1: Notification agent with learned threshold, fatigue penalty, hysteresis.

```

1 import numpy as np
2
3 class NotificationAgent:
4     def __init__(self, policy, max_per_day=3, refractory_h=4, persist=2, beta=0.25):
5         self.policy = policy # contextual bandit (Thompson sampling)
6         self.refractory_h, self.persist, self.beta = refractory_h, persist, beta
7         self.max_per_day = max_per_day
8         self.last_alert_t, self.streak, self.today_count = None, 0, 0
9
10    def crossing_prob(self, lo, hi, median, user_threshold):
11        """P(PM > threshold) from the calibrated interval (Gaussian approx of the band)."""
12        sigma = np.maximum((hi - lo) / 3.29, 1e-6) # 90% band ~ +/-1.645 sigma
13        from scipy.stats import norm
14        return float(norm.sf(user_threshold, loc=median, scale=sigma).max())
15
16    def decide(self, state, now):
17        p = self.crossing_prob(state.forecast_lo, state.forecast_hi,
18                               state.forecast_median, state.user_profile["threshold"])
19        ctx = self._context(state, now)
20        p_star = self.policy.threshold(ctx) # learned, per-user
21        if self.last_alert_t and hours_between(self.last_alert_t, now) < self.refractory_h:
22            p_star = min(1.0, p_star + self.beta) # fatigue penalty
23        self.streak = self.streak + 1 if p > p_star else 0
24        fire = (p > p_star and self.streak >= self.persist
25               and self.today_count < self.max_per_day)
26        if fire:
27            self.last_alert_t, self.today_count = now, self.today_count + 1

```

```
28         self.streak = 0
29         decision = {"alert": fire, "p": p, "p_star": p_star, "context": ctx}
30         state.alert_decision = decision
31         return decision
32
33     def feedback(self, ctx, reward):                                     # reward: +1 acted, 0 ignored, -1
34         muted                                                         muted
35         self.policy.update(ctx, reward)
```

Part IV

The Natural-Language Interface

Chapter 12

The LLM Layer

12.1 Role: the LLM is the interface, not the forecaster

A recurring temptation is to ask the language model to “predict the air quality.” We do not. The LLM cannot forecast $PM_{2.5}$ — it has no live data and no quantitative model — and asking it to would invite hallucination on a health-critical number. Instead the LLM does what it is good at: **understanding** a messy natural-language request, **calling** the right tools (the forecaster and planner agents) to get hard numbers, **retrieving** authoritative guidance, and **explaining** the result in plain, personalised language.

Key idea

Division of labour: *numbers come from the model; words come from the LLM*. Every quantitative claim in an answer must be traceable to a tool call or a retrieved document, never invented by the language model. This single rule is what makes a chatty interface safe for a health use case.

12.2 Tool use (function calling)

The LLM is given a set of *tools* described by JSON schemas (name, purpose, typed arguments). When the user asks a question, the model emits a structured request to call a tool with arguments it extracts from the query; the runtime executes the real function (the forecaster agent, the planner) and returns the result; the model then either calls another tool or composes the final answer. This is the standard function-calling / ReAct loop, and it is how the language layer *drives* the agents of Part III.

Listing 12.1: Tool schemas exposed to the LLM.

```
1 TOOLS = [  
2   {  
3     "name": "get_forecast",  
4     "description": "Probabilistic PM2.5 forecast for a location over a time window.",  
5     "parameters": {  
6       "type": "object",  
7       "properties": {  
8         "location": {"type": "string", "description": "City or 'lat,lon'"},  
9         "start": {"type": "string", "description": "ISO time, e.g. 2026-06-15T17:00"},  
10        "hours": {"type": "integer", "description": "Forecast horizon in hours"}  
11      },  
12      "required": ["location", "start", "hours"]  
13    }  
14  },  
15  {  
16    "name": "plan_route",
```

```

17     "description": "Lowest-exposure outdoor route/timing between two points.",
18     "parameters": {
19         "type": "object",
20         "properties": {
21             "origin": {"type": "string"}, "dest": {"type": "string"},
22             "window": {"type": "string"}, "activity": {"type": "string"}
23         },
24         "required": ["origin", "window", "activity"]
25     }
26 }
27 ]

```

Listing 12.2: The tool-calling loop (provider-agnostic sketch).

```

1 def answer_query(llm, user_msg, profile, tools, tool_impls, kb):
2     context = retrieve(kb, user_msg, profile, k=4)      # RAG (next section)
3     messages = [
4         {"role": "system", "content": SYSTEM_PROMPT},
5         {"role": "system", "content": f"User health profile: {profile}"},
6         {"role": "system", "content": f"Retrieved guidance:\n{context}"},
7         {"role": "user", "content": user_msg},
8     ]
9     for _ in range(MAX_STEPS):                          # bounded agent loop
10        resp = llm.chat(messages, tools=tools)
11        if resp.tool_calls:
12            for call in resp.tool_calls:
13                result = tool_impls[call.name](**call.args) # runs the real agent
14                messages.append({"role": "tool", "name": call.name,
15                                "content": json.dumps(result)})
16        else:
17            return resp.content                          # final grounded answer
18    return "Sorry, I could not complete that safely."    # loop guard

```

12.3 Retrieval-augmented generation (grounding)

The LLM’s free-text claims about health thresholds and advice must be grounded in real guidance (WHO 2021 guideline values, the national AQI category advisories, exercise recommendations for sensitive groups). *Retrieval-augmented generation* (RAG) supplies that: a curated knowledge base of guidance is chunked and embedded; at query time the most relevant chunks are retrieved and placed in the prompt, so the model paraphrases authoritative text instead of its own (possibly outdated) parametric memory.

The retrieval mathematics. Each chunk c_j and the query q are mapped to vectors by an embedding model $\mathbf{e}(\cdot) \in \mathbb{R}^D$. Relevance is cosine similarity,

$$\text{sim}(q, c_j) = \frac{\mathbf{e}(q)^\top \mathbf{e}(c_j)}{\|\mathbf{e}(q)\| \|\mathbf{e}(c_j)\|}, \quad (12.1)$$

and we retrieve the top- k chunks by this score (an approximate-nearest-neighbour index such as FAISS/HNSW makes this fast at scale). The retrieved chunks are concatenated into the Retrieved guidance block of Listing 12.2, with citations preserved so the answer can attribute its claims.

Listing 12.3: Minimal embedding retriever (cosine top-k).

```

1 import numpy as np
2
3 class GuidanceRetriever:
4     def __init__(self, embed_fn, chunks):
5         self.embed_fn = embed_fn
6         self.chunks = chunks

```

```

7     E = np.stack([embed_fn(c["text"]) for c in chunks])
8     self.E = E / (np.linalg.norm(E, axis=1, keepdims=True) + 1e-9)
9
10    def retrieve(self, query, k=4):
11        q = self.embed_fn(query)
12        q = q / (np.linalg.norm(q) + 1e-9)
13        sims = self.E @ q                                # cosine similarity, Eq. (cosine)
14        idx = np.argsort(-sims)[:k]
15        return [{"**self.chunks[i], "score": float(sims[i])} for i in idx]

```

12.4 Personalised risk explanation

The same forecast means different things to different people. AirSense AI conditions its explanation on a structured *health profile* — age band, asthma/COPD/cardiac conditions, pregnancy, planned activity intensity, and a self-reported sensitivity. The mapping from (forecast, profile) to a risk tier is *rule-based and transparent* (so it is auditable and defensible), and the LLM's job is only to *verbalise* that tier warmly and clearly and to attach the retrieved, cited advice.

Key idea

Keep the *risk logic* deterministic and inspectable (a small set of rules over the calibrated forecast and the profile). Let the LLM handle *language*, not *judgement*. This keeps the health-relevant decision boundary out of the stochastic component and makes the system testable.

Listing 12.4: Transparent risk tiering; the LLM only verbalises the result.

```

1 def risk_tier(forecast_median, forecast_hi, profile):
2     """Deterministic mapping -> tier + personal threshold. Auditable."""
3     base = forecast_hi      # be conservative: use the upper bound of the interval
4     # personal sensitivity multiplier (sensitive groups -> stricter effective limit)
5     sens = 1.0
6     if profile.get("asthma") or profile.get("cardiac"):
7         sens *= 0.6
8     if profile.get("age_band") in ("child", "senior"):
9         sens *= 0.7
10    if profile.get("pregnant"):
11        sens *= 0.7
12    if profile.get("activity") == "vigorous":    # exercise raises ventilation -> dose
13        sens *= 0.6
14    effective_limit = 60.0 * sens                # NAAQS 24h ref, tightened per person
15    if base <= effective_limit:
16        return {"tier": "ok", "limit": effective_limit, "value": base}
17    elif base <= 1.5 * effective_limit:
18        return {"tier": "caution", "limit": effective_limit, "value": base}
19    else:
20        return {"tier": "avoid", "limit": effective_limit, "value": base}

```

12.5 Prompt design, structured output, and guardrails

System prompt. The system prompt fixes the contract: answer only from tool results and retrieved guidance; never invent numbers; always state the uncertainty; include the appropriate medical disclaimer; and refuse out-of-scope medical questions, redirecting to a professional.

Structured output. For UI rendering and for testing, we ask the model to return a structured object (a recommendation tier, the numeric forecast with its interval, the suggested action, and the citations) and a

short prose summary, rather than free text only. Structured output makes the answer machine-checkable against the tool results — a guardrail in itself.

Caution

Health advice from an automated system carries real risk. Mandatory safeguards: (1) every numeric claim must match a tool result (verify programmatically before display); (2) always surface the uncertainty — if the interval straddles a threshold, say “uncertain, consider waiting,” never a flat “it’s safe”; (3) include a clear, non-dismissable medical disclaimer and advise sensitive users to consult a clinician; (4) fail safe — if a tool errors or confidence is low, decline to give a definitive go-ahead. These are product requirements, not nice-to-haves.

Hallucination mitigation, recap. The architecture minimises hallucination by construction: numbers come from tools (function calling), facts come from retrieval (RAG), judgement comes from deterministic rules (Listing 12.4), and the LLM is confined to orchestration and language. Remaining risk is checked by the structured-output verification step.

Part V

Integration, Evaluation, and Deployment

Chapter 13

End-to-End Walkthrough and System Architecture

13.1 Tracing the running example

We trace “*Is it safe to run outside in Kanpur this evening?*” through the whole system, to show how the pieces compose.

1. **LLM parses intent.** The language layer recognises an activity-safety question, extracts `location = Kanpur`, `activity = run` (vigorous $\Rightarrow$ high ventilation), and `window = this evening`. It loads the user’s stored health profile.
2. **Tool call: forecast.** The LLM calls `get_forecast(Kanpur, 17:00, 5h)`. This invokes the *forecaster agent*, which has already (on its hourly schedule) pulled live AQI and weather and run the ST-GNN brain. It returns the median $\text{PM}_{2.5}$ per hour and the *calibrated* interval — say a median rising to $\sim 95 \mu\text{g}/\text{m}^3$ by 7 pm with a 90% interval of [78, 116].
3. **Deterministic risk tiering.** The transparent rule (Listing 12.4) uses the *upper* bound and the profile. For a healthy adult doing vigorous exercise, the effective limit tightens (higher ventilation); the upper bound exceeds it $\Rightarrow$ tier caution/avoid for the 6–7 pm slot.
4. **Tool call: planner.** Because the activity is flexible, the LLM calls `plan_route / best-departure`. The *route-planner agent* sweeps the evening window and finds that an 8 pm start on a riverside loop roughly halves the inhaled dose versus a 6 pm city-centre run.
5. **Notification cross-check.** Independently, the *notification agent* had already evaluated whether to proactively alert this user; the same crossing probability that drove its decision is reused here for consistency.
6. **Grounded, personalised answer.** The LLM retrieves the relevant exercise-and-AQI guidance (RAG), verifies every number against the tool outputs, and replies in plain language: not safe for a hard 6 pm run; if you want to run, go at 8 pm on the riverside loop (lower and clearing); here is the uncertainty; and a one-line disclaimer for sensitive individuals.

Key idea

Notice that *no single component answered the question*. The brain supplied calibrated numbers, the planner supplied the better time and route, the rules supplied the judgement, RAG supplied the authority, and the LLM supplied the language. That composition is the whole point of the agentic design.

13.2 Tech stack

Table 13.1: Reference technology stack. Choices favour Python and open tooling.

Layer	Components
Modelling	PyTorch (or PyTorch Geometric / DGL) for the ST-GNN; NumPy/Pandas for features
Probabilistic	quantile heads + a conformal wrapper (MAPIE-style or custom)
Serving	TorchScript/ONNX model behind FastAPI; batched, cached hourly
Agents	a state-graph orchestrator (LangGraph-style) over the shared-state object
Routing	OSMnx/NetworkX graph from OpenStreetMap; Dijkstra/A*
Notification	contextual-bandit policy (Thompson sampling); a scheduler/worker queue
LLM layer	a function-calling LLM via the provider API; FAISS/HNSW for RAG
Storage	time-series store for measurements/forecasts; a vector store for guidance
Ops	experiment tracking (MLflow/W&B), containerised deploy, metrics dashboards

Chapter 14

Evaluation Plan (whole system)

Evaluation is layered, matching the architecture.

1) Forecast quality (Chapter 7): MAE/RMSE and CRPS/coverage/sharpness under strict OOT, with per-horizon, per-season, and per-AQI-band breakdowns, against persistence, the v0 XGBoost, and a GRU. The graph and uncertainty ablations are the scientific contributions.

2) Decision quality. A good forecast that yields bad decisions is worthless, so we evaluate the agents on *decision* metrics. For the planner: realised dose reduction of the recommended route/time versus a naive choice, measured on held-out forecast days (and ideally against the realised, observed field). For the notification agent: precision/recall of alerts against a labelled set of “genuinely useful” events, alert rate (alerts/user/day), and, in a pilot, the mute/dismissal rate — the real-world proxy for fatigue.

3) LLM quality. Faithfulness (does every claim trace to a tool result or retrieved chunk?), ground-ness/citation accuracy, refusal correctness on out-of-scope or unsafe queries, and calibration of its hedging language (does it say “uncertain” exactly when the interval straddles a threshold?). These are scored on a curated query suite, partly with an LLM-as-judge rubric and partly by automatic checks against tool outputs.

4) End-to-end. A scenario test suite (like the walkthrough above) with expected tiers/actions, run in CI so regressions are caught.

Chapter 15

Deployment, Monitoring, and MLOps

15.1 Serving and freshness

The brain is served as a versioned artefact (model registry) behind the forecaster agent’s API. Forecasts are recomputed hourly and cached; the agents and LLM read the cache, so user-facing latency is dominated by the LLM call, not inference. Every served forecast is logged with its inputs, so it can be scored against the truth once observations arrive.

15.2 Drift and calibration monitoring

Caution

Air-quality regimes shift (seasons, policy interventions, new emission sources). A model trained last winter can silently degrade. We must monitor, not assume.

- **Data drift:** track input feature distributions vs. the training distribution (population stability index / KS test) and alert on shift.
- **Concept drift:** track rolling MAE/CRPS of live forecasts against realised observations; trigger investigation/retraining when skill decays.
- **Calibration drift:** track *realised* interval coverage on a rolling window; the adaptive-conformal layer (Listing 6.2) auto-corrects α_t , and the dashboard shows whether coverage holds near $1 - \alpha$.
- **Retraining:** scheduled (e.g. monthly) plus drift-triggered; new models are shadow-evaluated against the incumbent before promotion.

Chapter 16

Risks, Limitations, and Ethics

- **Health-advice liability.** The system influences decisions with health consequences. Mitigations are built in (Chapter 12): conservative use of the interval's upper bound, mandatory uncertainty disclosure, deterministic auditable risk rules, clear disclaimers, and fail-safe refusal. The product positions itself as decision *support*, not medical advice.
- **Spatial coverage / fairness.** Monitoring stations are sparse and unevenly placed; interpolation is least reliable exactly in under-monitored (often under-served) neighbourhoods. We must report where confidence is low and avoid over-claiming there.
- **Over-trust in a clean number.** A confident-sounding answer can mask genuine uncertainty; honest hedging is a feature requirement, not a UX afterthought.
- **Feedback loops.** A notification policy optimised purely for engagement could learn to under-warn (fewer annoying alerts $\Rightarrow$ better short-term “reward”). The reward must encode genuine usefulness and safety, not just non-muting.
- **Privacy.** Health profiles and location are sensitive; store the minimum, keep processing local where possible, and be explicit about data use.

Chapter 17

Project Plan, Milestones, and Team

Table 17.1: Phased plan. Each phase has a concrete, demonstrable deliverable.

Phase	Theme	Deliverable
P0	Data & baselines	Live ingestion stub; OOT splits; persistence + v0 XGBoost + GRU baselines reproduced
P1	Spatio-temporal brain	STGCN & Graph WaveNet implemented; graph ablation; best point model chosen
P2	Calibrated uncertainty	Quantile head + CQR/adaptive conformal; CRPS/-coverage report
P3	Forecaster agent	Scheduled live forecasting service with caching, fallback, logging
P4	Route-planner agent	Dose-minimising routing & best-departure over the city graph
P5	Notification agent	Decision-theoretic, bandit-learned, fatigue-aware alerting
P6	LLM interface	Tool-calling loop, RAG grounding, personalised risk tiering, guardrails
P7	Integration & eval	End-to-end walkthrough, scenario tests in CI, monitoring dashboards
P8	Pilot & write-up	Small user pilot; ablation results; report/paper

Team and mentorship. The work parallelises cleanly into the three tracks — brain (P1–P2), agents (P3–P5), and language/integration (P6–P7) — which suits a mentored team: mentees can own a track end to end while sharing the common state contract (Listing 8.1) and evaluation harness. The brain track is the critical path; the agent and language tracks can develop against a stubbed forecaster until P3 lands.

Chapter 18

Conclusion

AirSense AI turns a $\text{PM}_{2.5}$ forecaster into something a person can actually use: a system that perceives live conditions, reasons about a specific user’s exposure under calibrated uncertainty, plans cleaner outdoor activity, decides when an alert is worth sending, and explains all of it in plain, grounded language. The technical spine is a spatio-temporal graph neural network with a calibrated probabilistic head; the product spine is a small set of cooperating agents driven by a tool-using, retrieval-grounded LLM; and the discipline throughout is honesty about uncertainty. The forecasting model is, quite literally, the brain of the loop — and the contributions that make this a project rather than a demo are the graph-vs-baseline and calibration experiments on the brain, the decision-theoretic and dose-based formulations of the agents, and the safety-first design of the language layer.

Bibliography

- [1] T. N. Kipf and M. Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR, 2017.
- [2] P. Veličković et al. *Graph Attention Networks*. ICLR, 2018.
- [3] M. Defferrard, X. Bresson, P. Vandergheynst. *Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering (ChebNet)*. NeurIPS, 2016.
- [4] B. Yu, H. Yin, Z. Zhu. *Spatio-Temporal Graph Convolutional Networks (STGCN)*. IJCAI, 2018.
- [5] Y. Li, R. Yu, C. Shahabi, Y. Liu. *Diffusion Convolutional Recurrent Neural Network (DCRNN)*. ICLR, 2018.
- [6] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang. *Graph WaveNet for Deep Spatial-Temporal Graph Modeling*. IJCAI, 2019.
- [7] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
- [8] S. Hochreiter and J. Schmidhuber. *Long Short-Term Memory*. Neural Computation, 1997.
- [9] K. Cho et al. *Learning Phrase Representations using RNN Encoder–Decoder (GRU)*. EMNLP, 2014.
- [10] S. Bai, J. Z. Kolter, V. Koltun. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling (TCN)*. arXiv:1803.01271, 2018.
- [11] R. Koenker and G. Bassett. *Regression Quantiles*. Econometrica, 1978.
- [12] V. Vovk, A. Gammerman, G. Shafer. *Algorithmic Learning in a Random World (Conformal Prediction)*. Springer, 2005.
- [13] J. Lei et al. *Distribution-Free Predictive Inference for Regression*. JASA, 2018.
- [14] Y. Romano, E. Patterson, E. Candès. *Conformalized Quantile Regression (CQR)*. NeurIPS, 2019.
- [15] I. Gibbs and E. Candès. *Adaptive Conformal Inference Under Distribution Shift*. NeurIPS, 2021.
- [16] T. Gneiting and A. E. Raftery. *Strictly Proper Scoring Rules, Prediction, and Estimation*. JASA, 2007.
- [17] E. W. Dijkstra. *A Note on Two Problems in Connexion with Graphs*. Numerische Mathematik, 1959.
- [18] P. E. Hart, N. J. Nilsson, B. Raphael. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths (A*)*. IEEE Trans. SSC, 1968.
- [19] D. Russo et al. *A Tutorial on Thompson Sampling*. Foundations and Trends in ML, 2018.

- [20] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
- [21] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.

Appendix A

Notation

Symbol	Meaning
N	number of monitoring stations (graph nodes)
F	number of input feature channels per station
P, Q	look-back window length, forecast horizon
$X^{(t)} \in \mathbb{R}^{N \times F}$	network feature matrix at time t
$Y^{(t)} \in \mathbb{R}^{Q \times N}$	target PM _{2.5} over the horizon
$G = (\mathcal{V}, \mathcal{E})$	station graph; A its adjacency
$\hat{A}$	symmetrically normalised adjacency, Eq. (5.4)
L	graph Laplacian
f_θ	forecasting model (the brain), Eq. (2.2)
ρ_τ	pinball loss at quantile τ , Eq. (6.2)
$\hat{q}$	conformal interval radius, Eq. (6.4)
α	target miscoverage ($1 - \alpha = \text{coverage}$)
p^*	Bayes-optimal alert threshold, Eq. (11.1)
V	minute-ventilation rate (dose model), Eq. (10.1)
$w_e(t)$	dose cost of routing edge e at time t , Eq. (10.2)